Fundamentals of Artificial Intelligence - Lecture 02

Session context

this 69-page deck and one 90-minute recording form a composite: an opening bias and utility block, Lecture 02 on agents, then the Lecture 03 Search block beginning at slide 30. Internal title pages mark these content boundaries.

Lecture overview

Lecture 02 establishes the theoretical foundations of Artificial Intelligence by defining rational agents as entities that perceive their environment and act to maximize expected utility. The lecture highlights significant challenges in this framework, particularly regarding algorithmic bias and the difficulty of defining a single "correct" utility function amidst conflicting societal norms, noting that creating a completely unbiased algorithm is impossible due to the lack of global consensus on moral decisions. While the course scope focuses specifically on reasoning by search and knowledge representation, excluding topics like deep learning and robotics, it acknowledges broader ethical implications through a scheduled guest lecture on AI's impact on human rights and democracy.

The technical core of the lecture introduces on-demand tree search as a method for exploring state spaces by expanding nodes from a frontier only when selected. A critical distinction is drawn between the finite underlying state-space graph and the potentially infinite search tree, which risks revisiting states without explicit cycle detection. To address this, the `GENERIC-SEARCH` algorithm is presented, utilizing both a frontier for available nodes and a closed set to track visited states, effectively functioning as graph search. These concepts are illustrated through standard problem domains such as route finding in Romania and the Pac-Man agent, where the primary challenge lies in determining the specific strategy for selecting which node from the frontier to expand next.

Slide 1: Bias in algorithms, in data, in use...

This slide introduces the concept of bias as a multifaceted issue occurring within algorithms, datasets, and practical applications. It highlights that bias is not limited to a single stage but can manifest across the entire lifecycle of algorithmic systems, from design and data collection to deployment and usage.

Slide 2: What would an ‘unbiased’ algorithm be?

This composite recording begins by revisiting the closing question from Lecture 01: what should an algorithm’s utility represent, and who chooses the standard? The opening block covers those final seven slides before the lecture turns to agents and search; the recording boundary does not change the slide-deck pairing.

Slide 3: Maximize Your

Expected utility combines the utility of each possible outcome with the probability of that outcome. The decision rule selects an available action with the highest probability-weighted expected value under the chosen utility function.

Slide 4: Utility?

The slide contrasts Go, where legal outcomes and game utility are explicit, with a household robot task whose success measure is harder to define. The contrast shows why a rational agent needs a specified objective before it can optimize.

Slide 5: Slide 5

The racing-game screenshot is discussed as an example of utility and proxy measures. An agent can raise a measured score in ways that miss the real objective, so the chosen utility should represent the outcome people actually value. The image does not reveal the game's exact reward mechanics.

Slide 6: What expected utility is right?

While this course uses unambiguous toy examples, AI agents maximizing expected utility have real-world implications where defining the correct utility function is difficult. Even with clear developer intent, agents may exhibit strange behaviors to achieve their goals, such as hacking a website to download data when standard methods fail. This highlights that optimizing expected utility requires ongoing conversation and awareness of how systems respond in extreme cases.

Slide 7: Guest lecture: Ethics in AI

A guest lecture on Ethics in AI will be held on Thursday, December 17, featuring Professor Nathalie A. Smuha from the University of Toronto Faculty of Law. Her research examines legal, ethical, and philosophical questions regarding AI's impact on human rights, democracy, and the rule of law. She is the author of Algorithmic Rule By Law and editor of the Cambridge Handbook on the Law, Ethics and Policy of Artificial Intelligence.

Slide 8: What would an 'unbiased' algorithm be?

Creating a completely unbiased algorithm is impossible because different societies hold conflicting norms regarding moral decisions, as demonstrated by the "World Machine" experiment on trolley problems. This lack of global consensus makes AI alignment difficult, particularly in determining which "expected utility" to prioritize when human values diverge across cultures. Consequently, defining what constitutes a correct or unbiased answer remains an unresolved challenge rooted in the subjective nature of humanity.

Slide 9: Further reading on implications of AI

Slide 9 provides a curated list of further reading on the implications of AI, including Stuart Russell's work on long-term futures and lethal autonomous weapons, alongside books by Russell and Nick Bostrom. The resources also cover Moshe Vardi's analysis of jobs, Asilomar panel discussions, research priorities from futureoflife.org, and reports from Stanford, the EU High Level Expert Group, and the Obama Whitehouse. Additionally, it references the 23 Asilomar AI principles for beneficial AI.

Slide 10: AI

This slide outlines key AI subfields, including rational agents, robots, machine learning, NLP, computer vision, and human-AI interaction. It distinguishes topics covered in this mandatory course-specifically reasoning by search and knowledge representation-from other mandatory or elective modules like deep learning, reinforcement learning, and AI ethics. The content serves as a curriculum map, listing specific course codes for related subjects such as robotics, speech recognition, and cognitive science that fall outside the current scope.

Lecture 02 - rational agents and environment types (source slides 11-29)

Slide 11: Fundamentals of Artificial Intelligence

This cover opens Lecture 02 on Agents. It begins a new lecture block within the same composite deck; the following slides define agents and classify their environments.

Slide 12: RECAP: Designing Rational Agents

An agent is an entity that perceives its environment through sensors and acts via actuators, while a rational agent specifically selects actions to maximize expected utility. The characteristics of percepts, the environment, and the action space dictate which techniques are appropriate for selecting these rational actions. This course focuses on general AI search techniques across various problem types, teaching students to recognize when existing methods can solve new problems, illustrated by building agents in environments like Pacman.

Slide 13: Environment types (1/2)

Environment types largely determine agent design through three key dimensions: observability, determinism, and dynamics. Environments are fully or partially observable; partial observability requires agents to maintain internal memory of past percepts. They are deterministic if the next state is fixed by current state and action, whereas stochastic environments involve potential action failures requiring contingency planning, with strategic determinism applying when only other agents act nondeterministically. Finally, static environments remain unchanged during deliberation, while dynamic ones change over time.

Slide 14: Environment types

The examples are classified by observability, determinism, and whether the environment changes while the agent acts. These properties affect what the agent must remember, predict, and plan for; the page lists examples but does not fully classify every one.

Slide 15: Slide 15

The chat screenshot shows a request for a healthy dinner suggestion with constraints such as the user profile and limited cooking time. It illustrates a text-based agent responding to context; it does not show the model’s internal reasoning or establish what information it lacks.

Slide 16: Environment types (2/2)

The environment type largely determines agent design across four key dimensions: discrete versus continuous states (e.g., chess versus robot movement), single versus multi-agent scenarios involving competitive or cooperative dynamics, known versus unknown physics requiring exploration, and known versus unknown performance measures necessitating human interaction. Continuous time environments require continuously operating controllers, while multi-agent settings may demand random behavior to avoid predictability. Unknown evaluation functions compel agents to observe and interact with a human principal to establish goals.

Slide 17: Environment types

Slide 17 classifies environments using criteria such as observability, determinism, and known physics across examples like Pacman, Backgammon, Taxi, and MS CoPilot Chat. Pacman is identified as partially observable and stochastic due to unpredictable ghost behavior, while Backgammon is fully observable despite involving dice. The lecture contrasts these with the Taxi environment, which is partially observable, and notes that real-world physics (like tire friction) are often unknown or difficult to model precisely.

Slide 18: Example agent designs

Slide 18 presents a conceptual framework for example agent designs, illustrating the core components and their interactions. The diagram identifies the central Agent, which connects to an external Environment through Sensors that provide Percepts and Actuators that execute Actions. This structure defines the fundamental loop where an agent perceives its surroundings and acts upon them.

Slide 19: Simple re'ex agents

A simple reflex agent maps the current percept, or a condition in that percept, directly to an action using condition-action rules. It does not use a history of prior percepts, so it works well only when the current percept contains the information needed for the decision.

Slide 20: Pacman agent program in Python

The `GoWestAgent` class extends the `Agent` base class to define a simple Pacman agent that prioritizes moving west. Its `getAction` method checks if `Directions.WEST` is available in the list of legal actions returned by `percept.getLegalPacmanActions()`. If west is legal, the agent returns `Directions.WEST`; otherwise, it returns `Directions.STOP`.

Slide 21: Simple re'ex agents

Simple reflex agents do not maintain a history of past states, making them viable only in fully observable environments. While their design is straightforward, they require rules that explicitly cover every possible perceptual case to function correctly.

Slide 22: Eat adjacent dot, if any

The Pac-Man example expresses a local condition-action rule: if a dot is adjacent, eat it. The line illustrates a simple reflex policy based on the current local percept; it does not define a complete strategy or reward function.

Slide 23: Eat adjacent dot, if any

This build continues the same rule. A reflex agent applies the condition to the current percept and selects the indicated action; without state or history, the rule cannot reason about a longer route or future consequences.

Slide 24: Pacman agent contd.

A reflex agent cannot behave well in all standard Pacman environments because the game is not fully observable, specifically regarding power pellet duration. While a lookup table could theoretically work if observability were complete, its size would be prohibitive. Furthermore, not every function can be implemented by an agent program due to computational limits, as illustrated by chess on slow hardware, NP-hard problems, and the halting problem.

Slide 25: Model-based agents

Model-based agents maintain an internal state updated by every new percept to track past events. They require a transition model to predict action outcomes and a sensor model to understand world evolution. For example, in Pac-Man, the transition model updates the agent's location and marks eaten dots as removed from the state representation.

Slide 26: Goal-based agents

Goal-based agents reason about achieving multi-step goals by utilizing their internal models. This approach distinguishes them from simpler agent types by focusing on future planning rather than immediate reactions. The lecture notes that most agents studied throughout the course fall into this category.

Slide 27: Goal-based Agents

Goal-based agents, also known as planning agents, make decisions by evaluating future action sequences to achieve a definite goal or multiple goals. These agents require an internal model of how the world evolves in response to actions. An optimal goal-based agent specifically aims to achieve its goal at the lowest possible cost.

Slide 28: Summary

An agent perceives and acts through an architecture and program, with a rational agent maximizing expected utility based on its percept sequence and environmental knowledge. Agent programs map percepts to actions via designs like reflex agents for immediate responses or goal-based agents for working toward objectives. Environments vary in difficulty, with partially observable, nondeterministic, dynamic, and multi-agent settings presenting the most significant challenges for agent design.

Slide 29: Precompute op.mal plan, execute it

Precomputing a plan and then executing it works when the modeled environment is sufficiently known and stable. If observations reveal that the world changed or the plan fails, the agent needs to replan rather than blindly continue.

Slide 30: Fundamentals of Artificial Intelligence

This cover marks the internal boundary from Lecture 02 on agents to Lecture 03 on search. The Library retains one composite PDF and one matching recording; pages 31 onward form the search block.

Slide 31: Content

This slide outlines the chapter's content, covering search problems and uninformed search methods including Depth-First Search, Breadth-First Search, and Uniform-Cost Search. Completing this material prepares students to solve questions 1 and 2 of project P1.

Slide 32: Search Problems

A search problem abstracts a task into an initial state, actions, a transition model, a goal test, and optionally a path cost. Search then finds an action sequence that reaches a goal within this representation.

Slide 33: Search Problems

A search problem comprises a state space, initial state, actions, transition model, goal test, and action cost function. Costs are calculated per step (+1), eating food (-10), winning (-500), dying (+500), or eating a ghost (-200). A solution is defined as an action sequence reaching a goal state, while an optimal solution minimizes the total accumulated cost of that sequence.

Slide 34: Search Problems Are Models

A search problem is a model of a task. It specifies an initial state, available actions and transitions, a goal test, and often a path-cost measure. The model deliberately keeps details needed to solve the task and omits details that do not affect the search objective.

Slide 35: What’s in a State Space?

A search state abstracts the world by retaining only details necessary for planning, such as coordinates and dot status. For pathing, states are (x,y) locations with a goal test checking destination arrival; for eating all dots, states include dot Booleans to verify if all are false. Transitions update these values based on actions like NSEW movement, while action costs quantify individual steps.

Slide 36: State Space Sizes

The full world state space combines agent positions (120), food configurations (2^{30}), ghost positions (12^2), and facing directions (4). However, efficient modeling reduces this significantly: pathfinding requires only the 120 agent locations, while the "eat-all-dots" problem requires tracking agent location and food status (120×2^{30}). This demonstrates that state space size depends on which variables are relevant to the specific problem being solved.

Slide 37: State Space Graphs and Search Trees

A state-space graph represents states and transitions, with a state appearing once even if several paths reach it. A search tree unfolds paths from the start, so the same state can appear more than once and cycles can create repeated branches. Search algorithms operate on the tree of explored paths while using the graph to detect repeated states.

Slide 38: State Space Graphs

A state space graph is a mathematical representation of a search problem where vertices denote abstracted world configurations and edges represent transitions labeled with actions. In this structure, each unique state occurs exactly once, distinguishing it from paths that may revisit states through different action sequences. While the full graph is often too large to build in memory, it serves as a useful conceptual model for applying goal tests to identify specific goal vertices within the set of possible states.

Slide 39: State Space Graphs

A state space graph is a mathematical representation of a search problem where vertices are abstracted world configurations and edges represent action results. The goal test identifies a set of goal nodes, and crucially, each state appears only once in this graph structure. Although the full graph is often too large to build in memory, it remains a useful conceptual model for understanding search problems.

Slide 40: Search Trees

A search tree represents a "what if" structure of plans and outcomes where the root node is the start state and children correspond to successors. Although nodes display states, they fundamentally represent specific plans, defined as sequences of states leading to those points. Because building the entire tree is often impossible, algorithms maintain a frontier of unexplored nodes, selecting elements to expand and checking if the goal state has been reached.

Slide 41: Search Trees through the State Space Graphs

Search trees are constructed on demand to minimize expansion, where each tree node represents an entire path in the state space graph rather than a single state. Consequently, reaching the same goal state via different sequences creates distinct nodes in the search tree, as the algorithm must track and score the full sequence of actions taken. This distinction ensures that unique solutions are preserved based on previous choices, unlike the single representation of states in the underlying graph.

Slide 42: On-demand tree search

On-demand tree search explores a state space by expanding nodes only when selected from the frontier. The algorithm initializes with the start node and iteratively selects a node to check for the goal condition; if not met, it generates child nodes from neighbor states and inserts them into the frontier. This process continues until a solution is found or the frontier becomes empty, returning failure if no path exists.

Slide 43: On-demand tree search

On-demand tree search explores a state graph by expanding nodes only when selected from the frontier. The algorithm initializes with the start node and iteratively selects a node to check for the goal condition; if not met, it generates child nodes from neighbor states and inserts them into the frontier. This process repeats until a solution is found or the frontier becomes empty, returning failure if no path exists.

Slide 44: On-demand tree search

On-demand tree search explores a state graph by maintaining a frontier of nodes, starting with the initial state. The algorithm repeatedly selects and removes a node from the frontier; if it is a goal state, the solution is returned. Otherwise, all neighboring states are generated as child nodes and inserted into the frontier until the frontier is empty or a solution is found.

Slide 45: The expanding frontier

A node in a search tree represents a path within the state-space graph, while the frontier (or fringe) is the set of all nodes available for expansion. Node expansion involves generating all successor nodes by considering available actions from the current state. Each newly generated successor becomes a child node added to the frontier, allowing the algorithm to systematically explore reachable states.

Slide 46: On-demand tree search

On-demand tree search explores a state space by maintaining a frontier of nodes, starting with the initial state. The algorithm repeatedly selects and removes a node from the frontier; if it is a goal state, the solution is returned. Otherwise, the algorithm generates child nodes for all neighbor states and inserts them into the frontier, continuing until the frontier is empty or a solution is found.

Slide 47: On-demand tree search

On-demand tree search explores a state space by maintaining a frontier of nodes, starting with the initial state. The algorithm repeatedly selects and removes a node from the frontier; if it is a goal state, the solution is returned. Otherwise, the algorithm generates child nodes for all neighbor states and inserts them into the frontier, continuing until the frontier is empty or a solution is found.

Slide 48: On-demand tree search

On-demand tree search explores a state space by initializing a frontier with the start node and iteratively selecting nodes for expansion. The algorithm checks if the current node's state is a goal; if not, it generates child nodes from neighbor states and inserts them into the frontier. This process repeats until a solution is found or the frontier becomes empty, returning failure if no path exists.

Slide 49: On-demand tree search

On-demand tree search explores a state space by maintaining a frontier of nodes, starting with the initial state. The algorithm repeatedly selects and removes a node from the frontier; if it is a goal state, the solution is returned. Otherwise, the algorithm generates child nodes for each neighbor state and inserts them into the frontier, continuing until the frontier is empty or a solution is found.

Slide 50: On-demand tree search

On-demand tree search explores a state space by maintaining a frontier of nodes, starting with the initial state. The algorithm iteratively selects and removes a node from the frontier; if it is a goal state, the solution is returned. Otherwise, all neighboring states are generated as child nodes and inserted into the frontier until the frontier is empty or a solution is found.

Slide 51: On-demand tree search

On-demand tree search explores a state graph by maintaining a frontier of nodes, starting with the initial state. The algorithm repeatedly selects and removes a node from the frontier; if it is a goal state, the solution is returned. Otherwise, the algorithm generates child nodes for each neighbor state and inserts them into the frontier until the frontier is empty or a solution is found.

Slide 52: On-demand tree search

On-demand tree search explores a state graph by maintaining a frontier of nodes, starting with the initial state. The algorithm repeatedly selects and removes a node from the frontier; if it is a goal state, the solution is returned. Otherwise, it generates child nodes for all neighbor states and inserts them into the frontier, continuing until the frontier is empty or a solution is found.

Slide 53: On-demand tree search

The `TREE-SEARCH` function performs an on-demand tree search by initializing a frontier with the start node and iteratively selecting nodes for expansion. It checks if a selected node's state is a goal; if not, it generates child nodes from neighbor states and inserts them into the frontier until a solution is found or the frontier is empty. This process uses a 4-state graph example to illustrate how nodes like `<S>` expand into children such as `<S, b>`.

Slide 54: On-demand tree search

On-demand tree search explores a state space by maintaining a frontier of nodes, starting with the initial state. The algorithm repeatedly selects and removes a node from the frontier; if it is a goal state, the solution is returned. Otherwise, the algorithm generates child nodes for all neighbor states and inserts them into the frontier, continuing until the frontier is empty or a solution is found.

Slide 55: On-demand tree search

On-demand tree search explores a graph by expanding nodes only when selected from the frontier. The algorithm initializes with the start node and iteratively selects a node to check for the goal state; if not found, it generates child nodes from neighbor states and inserts them into the frontier. This process continues until a solution is returned or the frontier becomes empty, resulting in failure.

Slide 56: On-demand tree search

On-demand tree search explores a graph by expanding nodes only when selected from the frontier. The algorithm initializes with the start state and iteratively selects a node to check for goal status; if not a goal, it generates child nodes via neighbor states and inserts them into the frontier. This process repeats until a solution is found or the frontier becomes empty, returning failure otherwise.

Slide 57: On-demand tree search

On-demand tree search explores a graph by maintaining a frontier of nodes, starting with the initial state. The algorithm repeatedly selects and removes a node from the frontier; if it is a goal state, the solution is returned. Otherwise, the algorithm generates child nodes for each neighbor state and inserts them into the frontier, continuing until the frontier is empty or a solution is found.

Slide 58: On-demand tree search

On-demand tree search explores a state space by maintaining a frontier of nodes, starting with the initial state. The algorithm iteratively selects and removes a node from the frontier; if it is a goal state, the solution is returned. Otherwise, the algorithm generates child nodes for each neighbor state and inserts them into the frontier, repeating until the frontier is empty or a solution is found.

Slide 59: Quiz: State Space Graphs vs. Search Trees

This slide presents a quiz asking for the size of the search tree generated from a specific 4-state graph starting at node S. The professor explains that while the state-space graph contains all states and transitions, the algorithm constructs a search tree where the start state serves as the root. In computer science, this tree is visualized with the root at the top, branching out to represent possible outcomes from the initial state.

Slide 60: Quiz: State Space Graphs vs. Search Trees

This slide presents a quiz comparing state space graphs and search trees using a specific 4-state graph containing nodes S, G, a, and b. It asks students to determine the size of the search tree generated from the start node S, highlighting that unlike the finite graph, the search tree can grow infinitely due to repeated states. The slide emphasizes this concept with the quote, "Those who don't know history are doomed to repeat it," illustrating how search trees fail to recognize previously visited nodes without explicit cycle detection.

Slide 61: func%on GENERIC-SEARCH(problem) # return a solu=on or failure

The `GENERIC-SEARCH` function implements an on-demand tree search with loop detection by maintaining a frontier of nodes and a closed set of visited states. It iteratively selects a node from the frontier, checks if it satisfies the goal condition, and adds unvisited neighbors to the frontier while tracking explored states to prevent cycles. Although termed "tree search" here, this algorithm functions as graph search in contexts like CS188 due to its ability to handle general graphs via the closed set mechanism.

Slide 62: Generic Search

Generic search maintains a frontier of discovered but unexpanded paths and repeatedly chooses one according to a strategy. Different algorithms vary this selection rule while sharing the same state-transition model.

Slide 63: Example: route-?nding in Romania

The Romania route-finding map turns a road network into a search problem: cities are states, roads are actions or edges, and distances provide step costs. Arad is the standard initial city and Bucharest the goal in the accompanying example. The map is a model for comparing search methods, not a full representation of travel.

Slide 64: Example: Traveling in Romania

The "Traveling in Romania" example defines a search problem where the state space consists of cities, starting from the initial state of Arad. Actions involve moving to an adjacent city, with the transition model ensuring arrival at that city and the goal test checking if the current state is Bucharest. The action cost corresponds to the road distance between cities, and the solution is a sequence of actions reaching the goal.

Slide 65: Crea=ng the search tree

The search tree grows by removing a path from the frontier, testing its endpoint, and adding paths to its successors. The frontier contains generated paths waiting for expansion.

Slide 66: Crea=ng the search tree

The explored set records states already expanded, while the frontier contains generated but unexpanded paths. Tracking both helps prevent repeated work when the state space contains cycles.

Slide 67: Crea=ng the search tree

This build completes the distinction between frontier and explored set. The algorithm expands one frontier path at a time and moves its state to explored before adding eligible successors.

Slide 68: Key concepts

The frontier or fringe comprises the set of all nodes currently available for expansion. Node expansion involves generating all successor nodes based on available actions, while the search strategy determines which specific node is expanded next. This selection mechanism represents the central question in defining a search algorithm.

Slide 69: RECAP: Pac-Man as an Agent

This slide recaps Pac-Man as an agent interacting with its environment through sensors and actuators to generate percepts and actions. While the full world state includes every detail like walls, score, and Pac-Man's orientation, the agent models only a relevant subset of this information. Specifically, details such as Pac-Man's facing orientation are irrelevant to the game's dynamics and are excluded from the agent's reasoning process.

Key points to remember

The technical core of the lecture introduces on-demand tree search as a method for exploring state spaces by expanding nodes from a frontier only when selected. A critical distinction is drawn between the finite underlying state-space graph and the potentially infinite search tree, which risks revisiting states without explicit cycle detection. To address this, the `GENERIC-SEARCH` algorithm is presented, utilizing both a frontier for available nodes and a closed set to track visited states, effectively functioning as graph search. These concepts are illustrated through standard problem domains such as route finding in Romania and the Pac-Man agent, where the primary challenge lies in determining the specific strategy for selecting which node from the frontier to expand next.